



UNIVERSITÄT
LEIPZIG

2. Tutorium: Git

Prof. Dr.-Ing. Norbert Siegmund
Tobias Matzke
Tristan Scholz

29. & 30.10.25



Organisatorisches

- Tutoren:
 - Tristan Scholz: scholz@studserv.uni-leipzig.de
 - Tobias Matzke: sg60licu@studserv.uni-leipzig.de
- Tutorien begleitend zu Übung und Vorlesung
- Ziel: Praktisches Anwenden der erlernten Konzepte
- Zeitslots: Wöchentlich Mi und Do jeweils 15:15-16:45, Hörsaal 19

1. Überblick Git

Was ist Git?

- Kostenfreies Open-Source Versionskontrollsystem
- Standard in der Softwareentwicklung
- Speichert benutzerdefiniert Schnappschüsse des Projekts, auf die später zurückgegriffen werden kann



Welche Probleme löst Git?

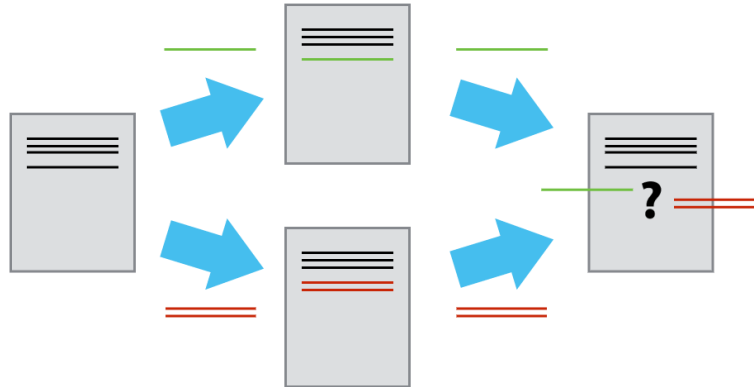
Name

📁 v 1.0

📁 v 1.1

📁 v 1.1a

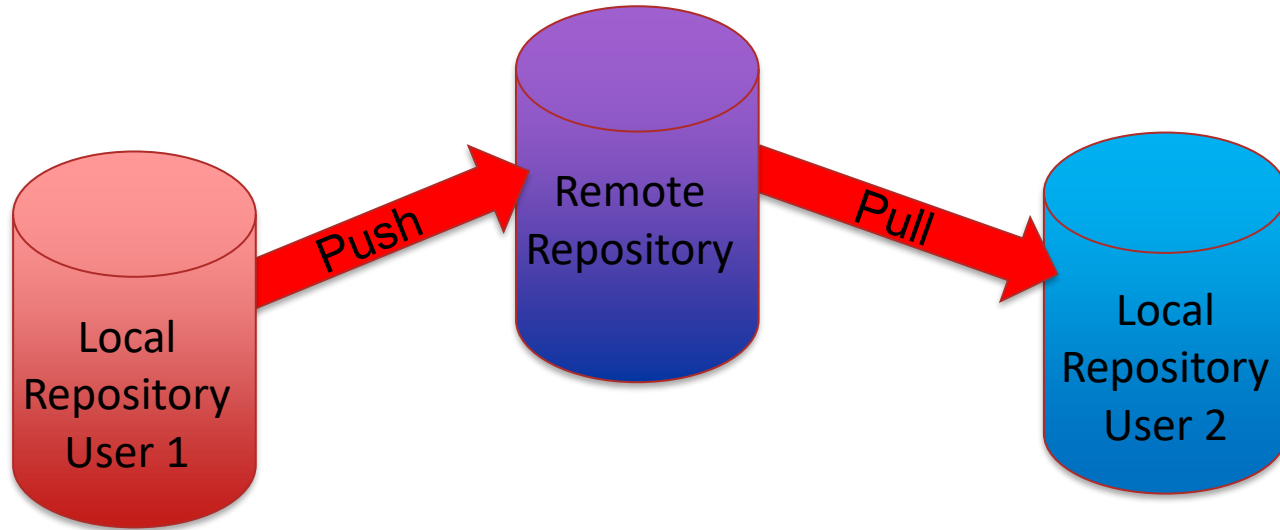
📁 v 2.0



shutterstock.com · 1553155220

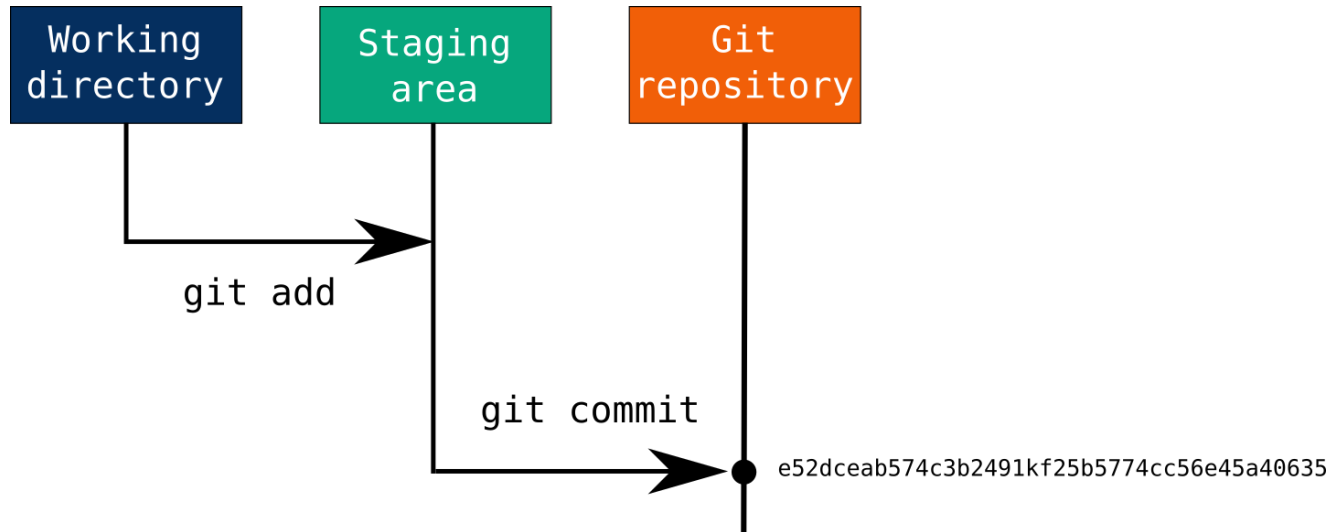
Wie funktioniert Git?

- Git speichert Schnappschüsse des Projekts im **Repository**
- NutzerInnen nehmen Änderungen zunächst im **lokalen** Repository vor
- Änderungen mehrerer NutzerInnen können in einem **remote** Repository zusammengeführt werden



Wie funktioniert Git?

- Änderungen werden von NutzerInnen in **Commits** im Repository gespeichert
- Dafür werden geänderte Dateien zunächst in der **Staging Area** abgelegt



<https://earthdatascience.org/workshops/intro-version-control-git/basic-git-commands/>

Was sind Commits?

- Ein **Commit** ist Schnappschuss des Projektes, der im Repository gespeichert wurde und alle Änderungen seit dem letzten Commit enthält
 - Kann bspw. Bug-Fixes, neue Features, Code-Cleanup, ... umfassen
- Commits zeichnen sich durch einen einzigartigen Hash und eine Commit Message aus, außerdem werden AuthorIn (Nutzername und E-Mail) sowie Timestamp vermerkt
- Alle Commits sind in der Git History einsehbar
 - Auf frühere Commits kann stets zurückgegriffen werden

2. Erste Schritte mit Git

Benutzte Befehle:

- **git version**
Gibt aktuell installierte Git-Version aus
- **git init**
Initialisiert ein leeres Repository im derzeitigen Ordner
- **git status**
Gibt alle veränderten Dateien im Working Directory sowie die Änderungen in der Staging Area an
- **git add <filename>**
Fügt die angegebene Datei der Staging Area hinzu. Mit „**git add .**“ werden alle Änderungen gestaged
- **git commit -m „Commit Message“**
Erstellt einen neuen Commit im Repository mit der angegebenen Commit Message und allen Änderungen in der Staging Area

Benutzte Befehle:

- **git diff**

Zeigt alle Änderungen im Working Directory an

- **git ls-files**

Listet alle von Git getrackten Dateien auf

- **git show <commit_hash>**

Zeigt Änderungen eines Commits an

- **git log**

Zeigt alle Commits in der Historie mit Message, Datum, Hash und Author an

Optionen (Beispiele):

- **-- oneline** - Kondensiert Commits für übersichtlichere Ansicht

- **-- graph** - Stellt Historie als Graph dar

- **-n 3** - Zeigt nur die 3 letzten Commits

.gitignore

- .gitignore-Datei dient als Liste von Dateien und Verzeichnissen, die Git nicht tracken soll
- Beispiel:

```
# Ignore notes.txt file
notes.txt

# Ignore entire logs directory
logs/

# Ignore all .class files
*.class
```

Best Practices für Commits

- Commit Message sollte kurz und bündig sein, aber klaren Überblick über Änderungen geben
 - Details können in Body der Commit Message geschrieben werden
 - Unterschiedliche Konventionen, z.B. Conventional Commits (<https://www.conventionalcommits.org/en/v1.0.0/>)

Negativbeispiel:

```
* ada217e Fixes
* 9c8b4d5 Some debug stuff
* 49fca46 Actually, that wasn't needed
* 12fba02 More Changes
* 5e133f0 Slight Changes
* 01808ba Typo
* 7e665d3 Documentation
* a852acd Started work
```

Besser:

```
bac1611 Add support for umlauts in usernames
61edb14 Fix typo in username input prompt
7947be1 Update README build instructions for version 2.0
```

Mehrzeilige Commit Message:

```
Add support for umlauts in usernames

Fix German umlauts 'ä', 'ö', and 'ü' being displayed as '?'
Update username input prompt accordingly
```

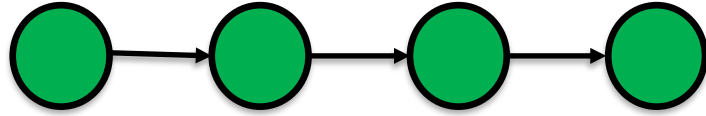
Best Practices für Commits

- Commits sollten logische Einheit bilden
 - Unabhängige Änderungen = separate Commits
 - Commits mit un- oder halbfertigen Änderungen vermeiden
 - Übersichtlichkeit: Nicht ein Commit pro Zeile, aber auch nicht hunderte Zeilen pro Commit

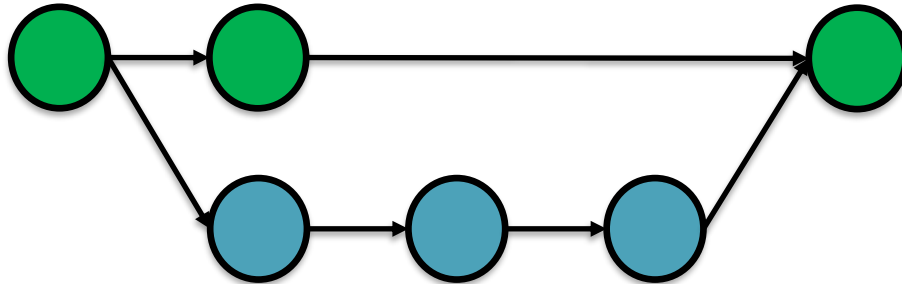
3. Branches

Was sind Branches?

- Bisher:

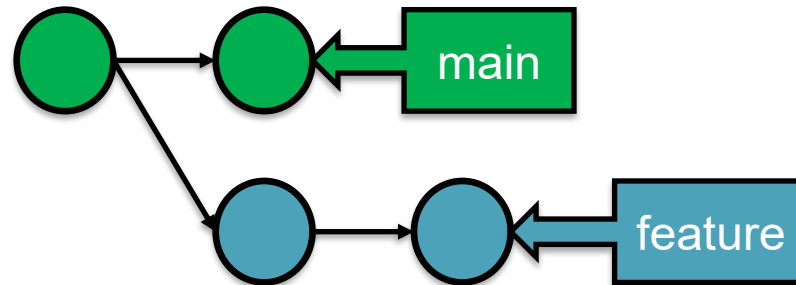


- Mit Branches:



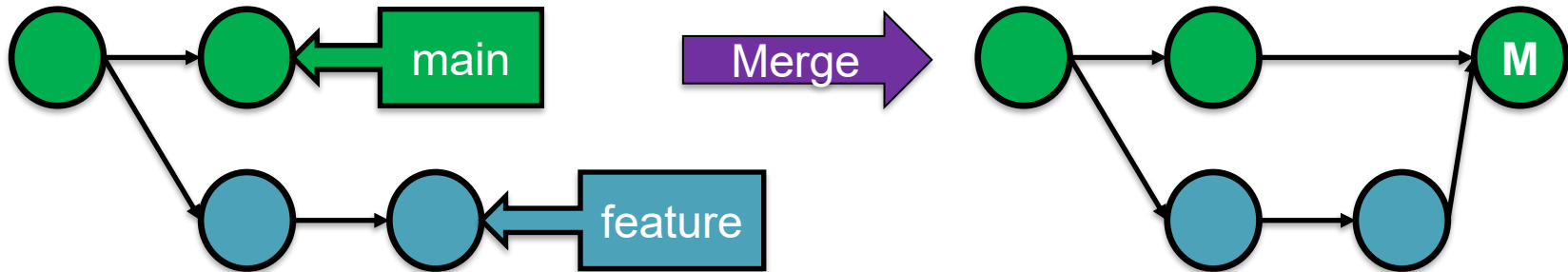
Was sind Branches?

- Branches stellen voneinander unabhängige Workspaces innerhalb eines Projektes dar
 - Neue Commits zunächst nur Teil des Branches, auf dem sich der Nutzer / die Nutzerin befindet
- Neues Repository wird mit **master**- bzw. **main**-Branch initialisiert
- Erstellen neuer Branches erlaubt das Committen von Änderungen, ohne den main-Branch zu beeinflussen
- EntwicklerInnen können frei zwischen Branches wechseln



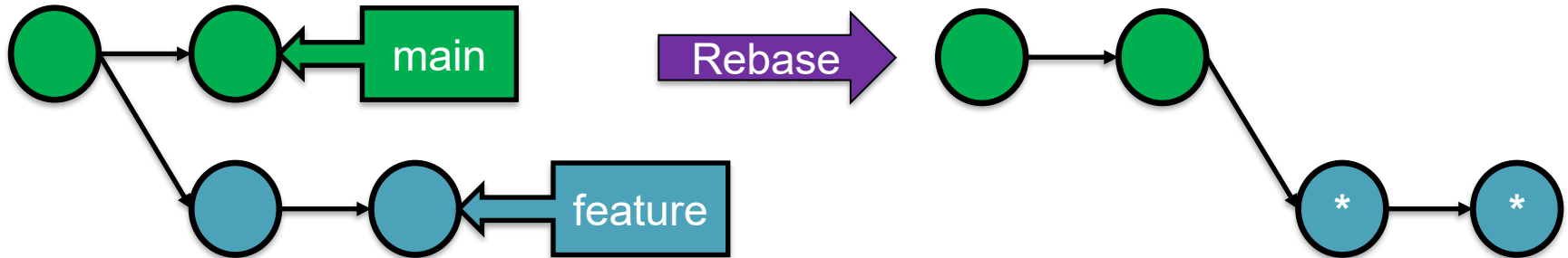
Wie werden Änderungen zusammengeführt?

- **Merge:** Fügt Änderungen in einem neuen Merge-Commit (M) zusammen
- „Merge preserves History“ – Vorige Commits der Branches bleiben unverändert
- Geeignet z.B. zum Updaten von main mit Änderungen anderer Branches
- Funktionsprinzip:



Wie werden Änderungen zusammengeführt?

- **Rebase:** Setzt Startpunkt des Branches um
- „Rebase rewrites History“ – Alte Commits werden durch neue (*) mit gleichem Inhalt ersetzt
- Geeignet z.B. zum Updaten kurzlebiger Feature-Banches mit Änderungen von main
- Funktionsprinzip:



Benutzte Befehle:

- **git branch**
Zeigt alle lokalen Branches an
- **git branch branch_name**
Erstellt einen neuen Branch mit dem Namen branch_name, ausgehend vom aktuellen Branch
- **git checkout branch_name**
Wechselt zum angegebenen Branch
- **git checkout -b branch_name**
Erstellt einen neuen Branch mit dem Namen branch_name und wechselt zum neuen Branch
- **git merge branch_name**
Führt einen Merge durch, der Änderungen des angegebenen Branches in den aktuellen Branch bringt
- **git branch -d branch_name**
Löscht den angegebenen Branch

Wann sollte man Branches anlegen?

- **Trunk-Based Development:** Entwicklung findet größtenteils direkt auf main statt
- **Feature-Branches:** Neue Features, komplexere Bug-Fixes, usw. werden in (oft kurzlebigen) Feature-Branches entwickelt und dann nach main gemerged
- **Git Flow:** Zusätzlicher develop-Branch zwischen main-Branch und Feature-Branches, main als stabile Release-Version genutzt

4. Remote Repositories

Was sind Remote Repositories?

- Repositories, die auf einem zentralen Server gehostet werden
- Neben Server meist weitere Funktionalitäten wie Issues, Merge Requests, Pipeline-Support, ...
- Bekannteste Plattformen:

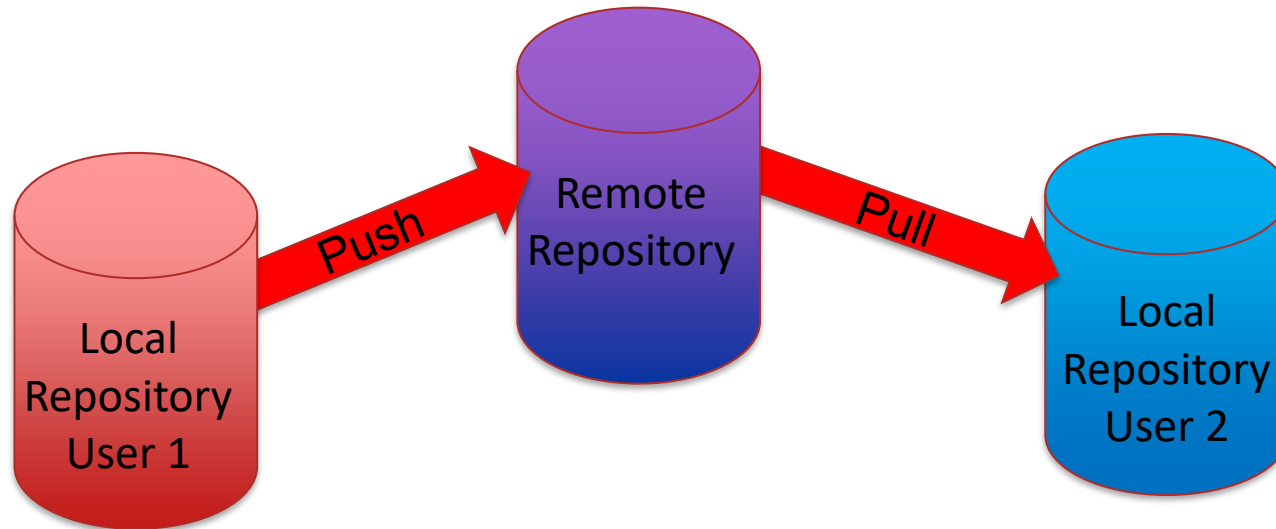


GitLab



Wie funktionieren Remote Repositories?

- Lokale Repositories können mit einem **remote** Repository über dessen URL verknüpft werden
- Standardbezeichnung für verknüpftes remote Repository: **origin**



Wie funktionieren Remote Repositories?

- Für Zugriff auf remote Repositories ist Authentifizierung erforderlich
- Möglich durch ständige Eingabe von Username und Passwort, besser: SSH-Keys
- Offizielle Anleitung zur Einrichtung eines GitLab SSH-Keys: <https://docs.gitlab.com/user/ssh/>

Benutzte Befehle:

- **git clone <repository_URL>**
Erstellt eine lokale Kopie eines remote Repositories
- **git remote add origin <repository_URL>**
Verknüpft ein bestehendes lokales Repository mit dem angegebenen remote Repository
- **git remote -v**
Zeigt remote Repository an, mit dem lokales Repository verknüpft ist
- **git pull origin <branch_name>**
Lädt neue Änderungen vom angegebenen Branch des remote Repositories
- **git push origin <branch_name>**
Veröffentlicht lokale Commits im angegebenen Branch des remote Repositories

Benutzte Befehle:

- **git push --set-upstream origin <branch_name>**
Führt einen Push zum angegebenen Branch durch und speichert diesen als Upstream-Branch des lokalen Branches --> Erlaubt Pushen und Pullen ohne spezifische Angabe eines Branches
- **git branch -r**
Listet alle Branches des remote Repositories auf
- **git checkout --track origin/<branch_name>**
Erstellt eine lokale Kopie des angegebenen Branches des remote Repositories

5. Was tun, wenn etwas schiefgeht?

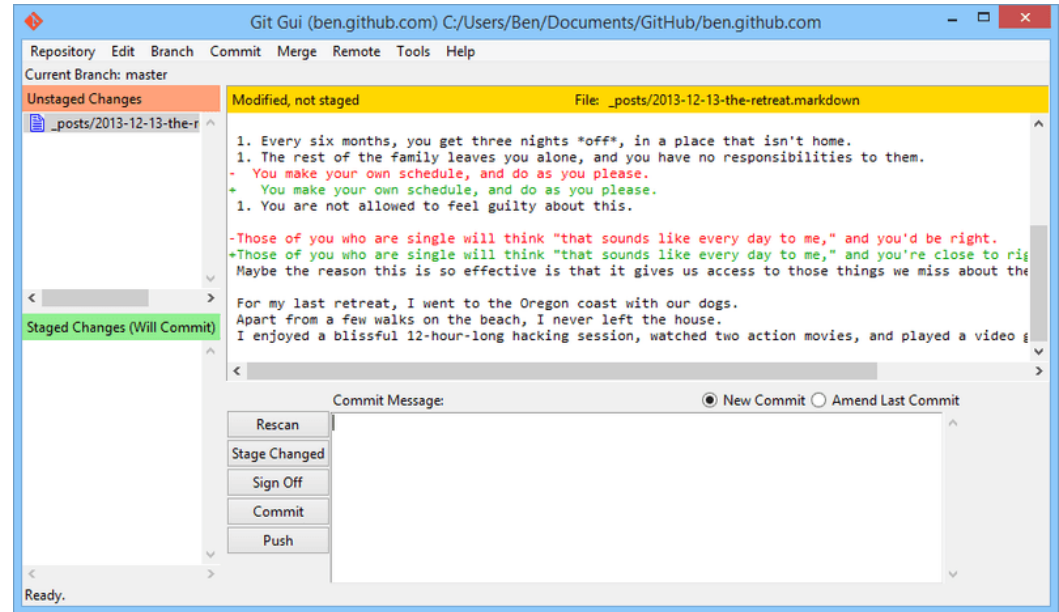
Benutzte Befehle:

- **git blame <filename>**
Zeigt für jede Zeile der angegebenen Datei, welcher Commit sie zuletzt geändert hat
- **git restore --staged <filename>**
Entfernt angegebene Datei aus der Staging Area, mit „**git restore -- staged .**“ werden alle Dateien entfernt
- **git restore <filename>**
Setzt alle Änderungen an der Datei, die noch nicht Teil eines Commits oder der Staging Area sind, zurück
- **git revert <commit_hash>**
Erstellt einen neuen Commit, der Änderungen des angegebenen Commits zurücksetzt
- **git commit --fixup <commit_hash>**
Erstellt einen Fixup-Commit mit Änderungen der Staging Area, bezogen auf den angegebenen Commit

6. GUIs

Git GUI

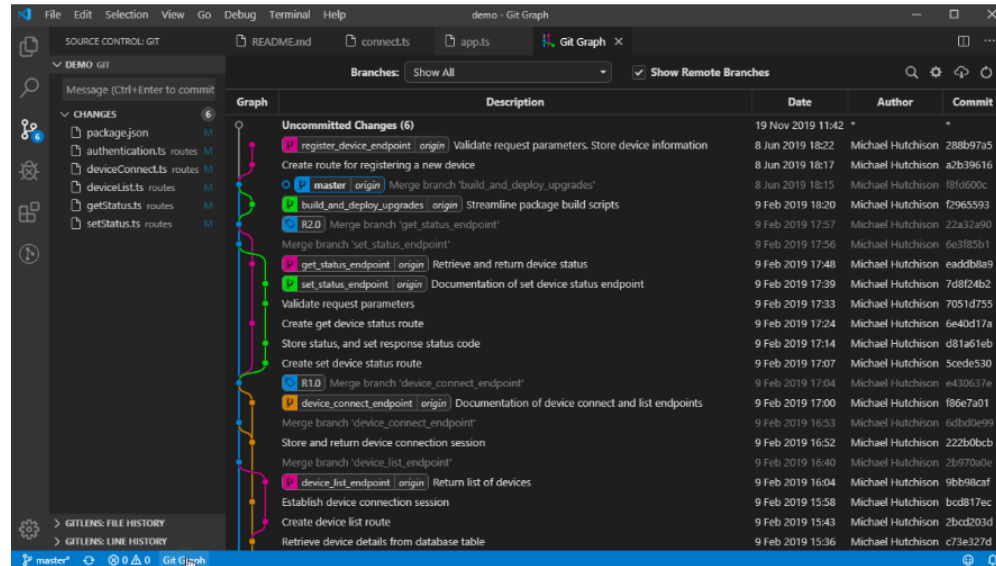
- Zusammen mit Git installiert
- Aufrufbar über den Befehl **git gui**
- Unterstützt v.a. beim Erstellen von Commits



<https://git-scm.com/book/de/v2/Anhang-A:-Git-in-anderen-Umgebungen-Grafische-Schnittstellen>

Visual Studio Code

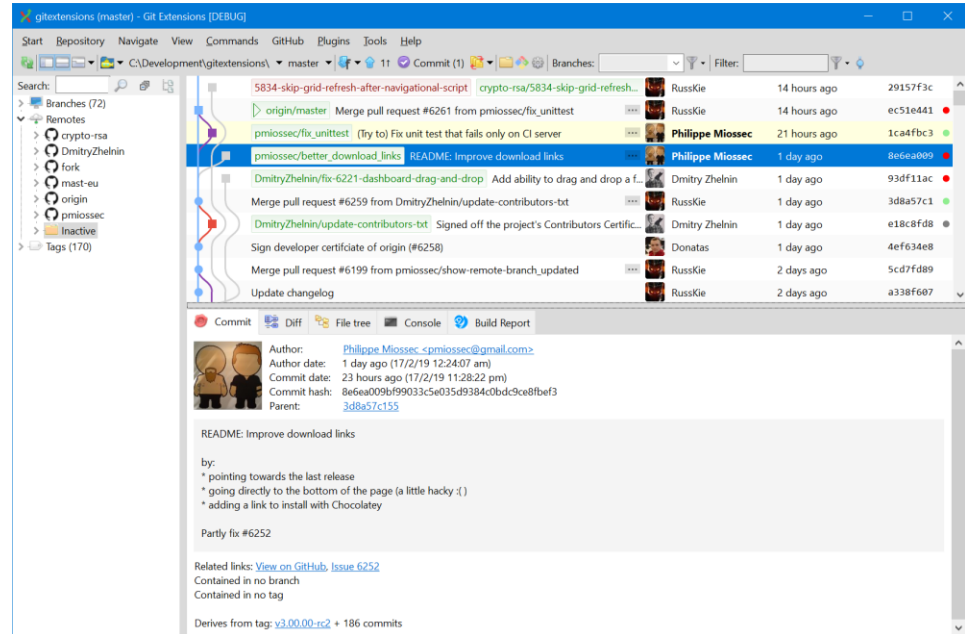
- Visual Studio Code bietet Unterstützung für grundlegende Git-Operationen
 - Kann durch Extensions noch verbessert werden, bspw. GitLens, Git Graph



<https://marketplace.visualstudio.com/items?itemName=mhutchie.git-graph>

Third-Party GUIs

- Übersicht von Third-Party GUIs auf offizieller Git-Seite: <https://git-scm.com/tools/guis>
- Bspw. Git Extensions
(<https://gitextensions.github.io/>)



Weitere Ressourcen

- Git Flight Rules: <https://github.com/k88hudson/git-flight-rules>
- Online Git Tutorials:
 - GeeksforGeeks: <https://www.geeksforgeeks.org/git/git-tutorial/>
 - Git Immersion: <https://gitimmersion.com/index.html>
- Offizielles Git Cheat Sheet: <https://git-scm.com/cheat-sheet>



UNIVERSITÄT
LEIPZIG

Fragen?

Tobias Matzke: sg60licu@studserv.uni-leipzig.de

Tristan Scholz: scholz@studserv.uni-leipzig.de